

Relatório Final do Projeto Prático

José Santarém Gomes

Curso de Técnicas de Hyperautomation

Resumo

Este relatório apresenta o desenvolvimento e os resultados de uma automação em Python para conferência de itens de importação. O projeto processa uma fila armazenada em CSV, compara quantidades informadas e encontradas, registra o resultado de cada item, trata divergências sem interromper o lote e impede o reprocessamento de itens já concluídos.

A massa de teste contém 12 itens. Na primeira execução, 8 itens foram concluídos e 4 foram classificados com erro de negócio por divergência de quantidade. Na segunda execução, nenhum item foi processado novamente: os 8 concluídos foram ignorados por idempotência e os 4 itens com erro permaneceram bloqueados para análise. Quatro testes automatizados também foram executados com sucesso.

Resultados principais

Indicador	Resultado
Itens da massa de teste	12
Concluídos na primeira execução	8
Erros de negócio na primeira execução	4
Itens reprocessados na segunda execução	0
Testes automatizados	4 aprovados

Palavras-chave: Hyperautomation; Python; fila de trabalho; idempotência; observabilidade; logging; testes automatizados.

1 Introdução

Processos baseados em listas ou lotes precisam continuar funcionando mesmo quando um registro apresenta erro. Também precisam fornecer evidências que permitam identificar o que foi processado, quando ocorreu e qual foi o resultado. Sem esses controles, uma nova execução pode duplicar operações ou esconder divergências importantes.

O projeto foi desenvolvido para demonstrar esses controles em uma solução pequena e independente de plataformas reais de orquestração. O arquivo `bot.py` funciona como ponto de entrada; um CSV representa a fila; um arquivo de log registra os eventos; e arquivos de evidência preservam os resultados das execuções.

2 Objetivos

2.1 Objetivo geral

Desenvolver uma automação reproduzível capaz de processar individualmente uma fila de itens de importação, validar quantidades, tratar falhas sem interromper o lote e produzir evidências do resultado.

2.2 Objetivos específicos

- Manter `bot.py` e `requirements.txt` na raiz do pacote publicável.
- Representar a fila com estados pendente, processando, concluído e erro.
- Impedir que itens concluídos sejam processados novamente.
- Registrar identificador, evento, severidade e timestamp para cada item.
- Emitir alerta somente quando um critério operacional for atingido.
- Comprovar o funcionamento por meio de duas execuções e testes automatizados.
- Permitir que a massa de dados seja restaurada para nova demonstração.

3 Cenário e massa de dados

O cenário simula a conferência de componentes associados a registros de importação. Cada item contém uma quantidade informada e uma quantidade encontrada. Quando os valores são iguais, o item é concluído. Quando são diferentes, o item recebe status de erro e uma mensagem com os valores comparados.

Tabela 1. Massa de dados utilizada

Item	Informada	Encontrada	Resultado esperado
IMP-001	10	10	Concluído
IMP-002	20	18	Erro
IMP-003	5	5	Concluído
IMP-004	12	12	Concluído

Item	Informada	Encontrada	Resultado esperado
IMP-005	8	6	Erro
IMP-006	15	15	Concluído
IMP-007	30	30	Concluído
IMP-008	24	22	Erro
IMP-009	9	9	Concluído
IMP-010	18	17	Erro
IMP-011	7	7	Concluído
IMP-012	40	40	Concluído

4 Estrutura da solução

```
projeto_hyerautomation/
├── bot.py
├── reset_fila.py
├── requirements.txt
├── README.md
├── data/
│   ├── fila_itens.csv
│   └── fila_itens_inicial.csv
├── evidencias/
├── logs/execucao.log
└── tests/test_bot.py
```

O arquivo bot.py contém a lógica principal. O arquivo fila_itens.csv representa a fila ativa. O arquivo fila_itens_inicial.csv mantém a massa original, enquanto reset_fila.py restaura a fila para uma nova demonstração. O diretório evidencias preserva as saídas e o diretório logs contém o histórico estruturado.

4.1 Campos da fila

Campo	Finalidade
item_id	Identificador único do item
descricao	Descrição do componente
quantidade_informada	Valor esperado pelo processo
quantidade_encontrada	Valor efetivamente conferido
status	Estado atual do item
tentativas	Quantidade de processamentos iniciados
mensagem	Resultado ou motivo do erro

4.2 Estados e transições

Estado	Significado	Transição esperada
pendente	Aguardando processamento	processando
processando	Item reservado pelo bot	concluído ou erro
concluído	Validação realizada com sucesso	Estado final
erro	Divergência ou dado inválido	Aguarda correção

5 Funcionamento do bot

- 1. Carregar o CSV e validar seu cabeçalho.
- 2. Selecionar somente itens com status pendente.
- 3. Alterar o item para processando e aumentar tentativas.

- 4. Salvar a fila antes da validação do item.
- 5. Converter e comparar as quantidades.
- 6. Marcar o item como concluído ou erro.
- 7. Registrar o evento no log e salvar novamente a fila.
- 8. Continuar com o próximo registro, mesmo após uma falha.
- 9. Avaliar o critério de alerta e emitir o resumo final.

6 Idempotência e tratamento de erros

A idempotência foi implementada pela consulta ao status antes do processamento. Itens concluídos são ignorados e não têm o campo tentativas incrementado. Itens com erro também permanecem bloqueados até que uma pessoa corrija os dados e autorize uma nova tentativa, alterando seu status para pendente.

Regra principal de seleção:

```
if status == "concluido":  
    ignorar item  
  
if status == "erro":  
    manter bloqueado  
  
if status == "pendente":  
    processar item
```

A gravação utiliza um arquivo temporário e a operação `os.replace`. Esse procedimento reduz o risco de deixar o CSV parcialmente gravado. Cada item também é protegido por tratamento de exceções, permitindo que um erro de validação ou uma falha inesperada seja registrado sem encerrar o lote.

7 Observabilidade e alerta

7.1 Logging estruturado

Os eventos são armazenados em `logs/execucao.log` no formato JSON Lines, com um objeto JSON por linha. Cada registro contém timestamp em UTC, nível de severidade, identificador do item, evento e mensagem.

```
{"timestamp": "2026-09-10T06:04:06Z",  
  "nivel": "WARNING",  
  "item_id": "IMP-002",  
  "evento": "DIVERGENCIA_ENCONTRADA",  
  "mensagem": "Quantidade informada: 20; quantidade encontrada: 18."}
```

7.2 Critério de alerta

O alerta simulado é emitido quando dois ou mais itens apresentam erro na mesma execução. O critério evita interromper o responsável por uma ocorrência isolada e sinaliza uma concentração de falhas que exige verificação humana. Como a primeira execução encontrou quatro divergências, foi produzido um evento ALERTA_CRITICO. Na segunda execução nenhum novo erro foi processado e, portanto, nenhum novo alerta foi gerado.

8 Testes automatizados

Os testes foram implementados com unittest, biblioteca nativa do Python. A fila do teste de idempotência é criada em diretório temporário, evitando qualquer alteração na massa principal.

Teste	Comportamento verificado	Resultado
test_item_com_quantidades_iguais	Quantidades iguais geram status concluído	Aprovado
test_item_com_divergencia	Valores diferentes geram status erro	Aprovado
test_quantidade_negativa	Valor negativo é rejeitado	Aprovado
test_idempotencia	Segunda execução não aumenta tentativas	Aprovado

A execução apresentou quatro testes aprovados e resultado final OK. O arquivo evidencias/resultados_testes.txt preserva essa saída.

9 Resultados das execuções

Tabela 2. Comparação das duas execuções

Indicador	Primeira execução	Segunda execução
Itens disponíveis	12	12
Itens processados	12	0
Itens concluídos na execução	8	0
Itens com erro na execução	4	0
Ignorados por conclusão anterior	0	8
Bloqueados com erro anterior	0	4
Alerta crítico	Emitido	Não emitido

Na primeira execução, os itens IMP-001, IMP-003, IMP-004, IMP-006, IMP-007, IMP-009, IMP-011 e IMP-012 foram concluídos. Os itens IMP-002, IMP-005, IMP-008 e IMP-010 apresentaram divergências. O processamento chegou ao final mesmo com quatro itens em erro.

Na segunda execução, os 8 itens concluídos foram ignorados e os 4 itens em erro permaneceram bloqueados. O campo tentativas continuou com valor 1, comprovando que nenhuma operação anterior foi repetida.

10 Procedimento de reprodução

A reprodução deve ser feita na raiz do projeto, com Python 3.10 ou superior.

Etapas	Comando
Restaurar a massa	<code>python reset_fila.py</code>
Primeira execução	<code>python bot.py</code>
Segunda execução	<code>python bot.py</code>
Executar testes	<code>python -m unittest discover -s tests -p "test_*.py" -v</code>

Após a primeira execução, deve-se observar 8 itens concluídos e 4 erros. Sem restaurar a fila, a segunda execução deve apresentar zero itens processados, 8 ignorados por idempotência e 4 bloqueados com erro anterior.

11 Evidências produzidas

- logs/execucao.log: eventos estruturados de início, processamento, conclusão, divergência, alerta e encerramento.
- evidencias/primeira_execucao.txt: resumo da execução completa dos 12 itens.

- evidencias/segunda_execucao.txt: comprovação de que nenhum item foi reprocessado.
- evidencias/fila_apos_primeira_execucao.csv: resultado individual de cada item.
- evidencias/resultados_testes.txt: saída dos quatro testes automatizados.
- data/fila_itens_inicial.csv: massa original utilizada para reprodução.

12 Limitações

A solução foi intencionalmente mantida local e simples. O CSV não oferece o mesmo controle de concorrência de um banco de dados e, por isso, o projeto pressupõe somente uma instância do bot em execução. O alerta é simulado no terminal e no log, sem envio real de e-mail. Também não há integração com navegador, API externa ou plataforma de orquestração, pois esses recursos não são necessários para demonstrar os objetivos do exercício.

13 Conclusão

O projeto atendeu ao objetivo de demonstrar uma automação estruturada, reproduzível e observável. A fila foi processada item por item; divergências foram registradas sem interromper o lote; o alerta foi emitido segundo um critério explícito; e a segunda execução comprovou a idempotência.

Os arquivos de log, os resultados das execuções, o CSV processado e os testes automatizados formam uma cadeia de evidências coerente com o comportamento relatado. A massa inicial e o script de reinicialização permitem repetir a demonstração em outra máquina sem acesso a um orquestrador real.